
ARCHITECTURES
SERVICE MODELS
DEPLOYMENT MODELS
TARGET
A FICTIONAL BREACH

From Architectures to Attacks

From architectures to attacks

Why the first half matters for the second

- The **attack surface** is every point where an attacker could try to get in or take data out: accounts, services, devices, software updates, suppliers and people.
- Architecture decides the **attack surface**: every era added ways in, from terminals and PCs to networks, remote vendor access and cloud consoles.
- Architecture decides the **trust boundaries**: once inside, what can an account or a system reach?
- Architecture decides the **evidence**: which logs exist, where they are kept, and who can read them.
- Attacks land on the **legacy** and on the **joins**: unpatched PCs (WannaCry), management tools (SolarWinds), third parties (Target).

Target 2013

2013 Target attack

- **What happened**: attackers stole customer data from Target's **point of sale** terminals during the busiest shopping period of the year.
- An estimated **40 million** payment cards and **70 million** customer records.
- **$61 million** of breach costs in the last quarter of 2013 alone ($17 million after insurance), in a quarter when profit fell **46%**.
- The way in: credentials belonging to a **third-party HVAC contractor** with remote access.

Target (2014) Target reports fourth quarter and full-year 2013 earnings.
https://corporate.target.com/press/release/2014/02/target-reports-fourth-quarter-and-full-year-2013-earnings

How it unfolded
2013

When	What happened
By September	Malware in emails infects the HVAC contractor, Fazio Mechanical, and its credentials for Target are stolen.
12 November	The attackers first get into Target's network.
15 to 30 November	Malware is installed on point of sale terminals, and by 30 November most are infected.
28 Nov to 3 Dec	Antivirus and FireEye tools raise alerts, which are not acted on.
2 to 18 December	About 11 GB of stolen data is sent to external servers, one of them in Russia.
12 December	The US Department of Justice tells Target.
19 December	Target announces the breach.

US Senate Committee on Commerce, Science and Transportation (2014) A "Kill Chain" Analysis of the 2013 Target Data Breach.
https://www.commerce.senate.gov/wp-content/uploads/media/doc/2014%200325%20Target%20Kill%20Chain%20Analysis.pdf

Target
Lessons learned

- Enterprises are complex systems of interconnected elements: **weak security in a subsystem** harms the whole.
- Attacks take **many steps** over weeks, and each step is an opportunity to notice.
- The costs of not addressing security are significant, both in the incident and in **rebuilding the business**.


ARCHITECTURES
SERVICE MODELS
DEPLOYMENT MODELS
TARGET
A FICTIONAL BREACH

A Fictional Breach

Aldermoor Provisions

A fictional example

- A fictional **food wholesaler** supplying restaurants and shops across the region, with a growing online ordering service.
- Customer accounts, orders and payment details are held in a **PostgreSQL database** (`stock`) on one Linux server, backed up every night.
- A **lift maintenance contractor** has remote access to the server, to monitor the building's lifts.
- In early December, Aldermoor's bank reports that cards used with Aldermoor are turning up in **fraud**.
- The server's logs for November are recovered. **What happened?**

Syslog

What is a log?

Start here

- A log is a computer's **diary**: each line records one event.
- Most lines answer four questions: **when**, on **which machine**, by **which program**, and **what happened**.
- Like a building's visitor book: useful only if it is **kept**, **complete** and **trusted**.
- Systems write thousands of lines a day, so they need a standard way to produce, move and store them. That standard is **syslog**.

Syslog

The common way systems record events

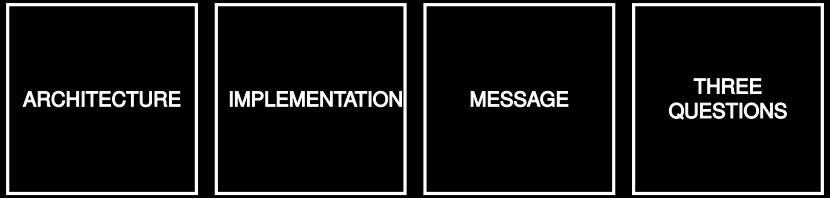
- Unix-like servers, routers, firewalls and many security products can **emit syslog**: one event, one message. (Windows keeps its own Event Log, and agents forward it.)
- The older BSD practice is described in RFC 3164 (2001), and the protocol was standardised in **RFC 5424** (2009).
- CyBOK treats system logs such as these as a core **data source** for security operations.

Gerhards (2009) RFC 5424: The Syslog Protocol. Debar (2021) CyBOK Security Operations and Incident Management Knowledge Area.
https://www.rfc-editor.org/rfc/rfc5424
https://www.cybok.org/media/downloads/Security_Operations_Incident_Management_v1.0.2.pdf

Words you will need

A short glossary

Term	Meaning
Program, process	A piece of software, and one running copy of it, with its own process ID
Daemon	A program that runs in the background, without anyone typing into it
Host, hostname	A computer on a network, and its name
Server	A computer, or program, that provides a service to others
Protocol	Agreed rules for how two computers exchange messages
Port	A numbered door on a computer, where a particular service listens
UDP	Send a message and forget it: no check that it arrived
TCP	Send a message and check that it arrived
TLS	TCP plus encryption and proof of identity, as in `https`
UTC	The world's reference time: no time zone, no summer time


ARCHITECTURE
IMPLEMENTATION
MESSAGE
THREE QUESTIONS

ARCHITECTURE

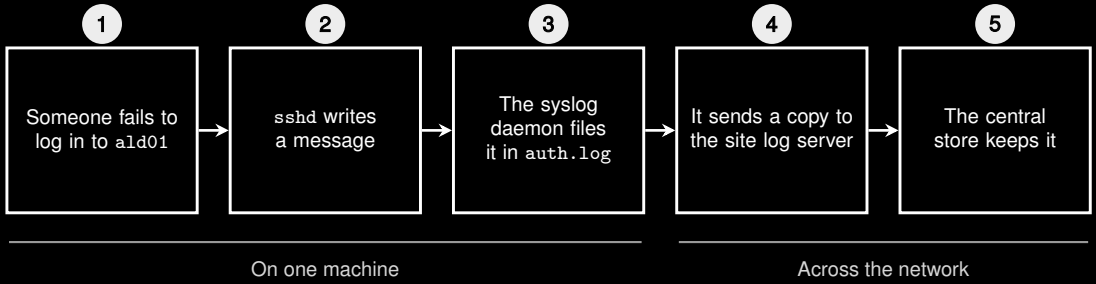
IMPLEMENTATION

MESSAGE

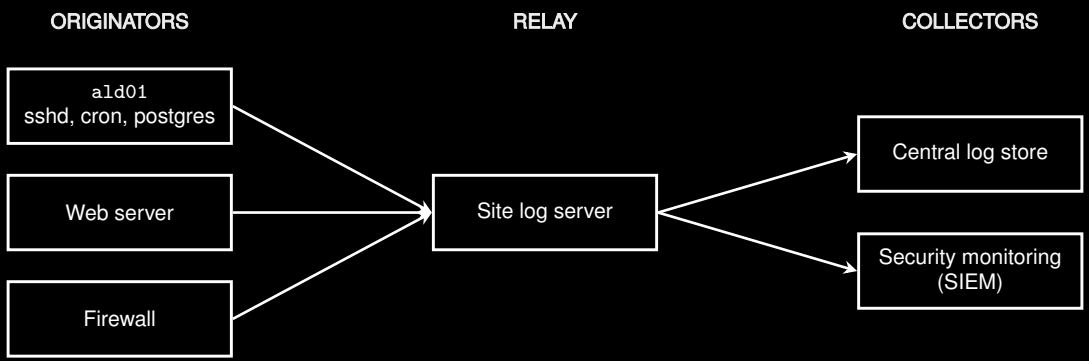
THREE QUESTIONS

Follow one message

From a failed login to the evidence store



Syslog architecture
Three roles



Syslog architecture
What each role does

- **Originator**: generates the message. A program, and the host it runs on.
- **Relay**: forwards messages from originators, or other relays, towards collectors.
- **Collector**: gathers messages for storage and analysis.
- One machine can play several roles, and one message can go to several collectors.
- Syslog is **simplex**: the originator gets no acknowledgement that a message arrived.

Gerhards (2009) RFC 5424: The Syslog Protocol.
`https://www.rfc-editor.org/rfc/rfc5424`

Syslog architecture

Three layers

Layer	What it does
Content	The information itself: what happened, where, and when.
Application	Generates, interprets, routes and stores messages: the syslog daemon.
Transport	Puts messages on the wire and takes them off: UDP, TCP or TLS.

Like sending a letter: the content is what you write, the application is the post office that sorts it, and the transport is the van that carries it.

Where the roles sit

Back to the architectures

- **On premises and IaaS**: you run the originators, and you choose the relays and collectors.
- **PaaS and SaaS**: the provider is the originator, and you receive what its logging service chooses to export.
- Relays often sit at **network boundaries**: a branch, a data centre, a cloud region. The path between them needs protecting.
- The **collector** is what an attacker most wants to blind: it holds the evidence from every machine.

Architecture
In short

- Programs on each machine **produce** messages.
- Relays **pass them on**, and collectors **keep** them.
- Messages flow **one way**, with no receipt.
- Where the collector sits, and who runs it, decides **who holds the evidence**.

ARCHITECTURE

IMPLEMENTATION

MESSAGE

THREE QUESTIONS

Implementation
The big picture

- Implementation answers four practical questions:
  - **Which program** does the work?
  - **How is it told** what to do?
  - **Where** does it send each message?
  - **How long** is each message kept?
- Think of an office post room: one clerk receives all the internal mail, sorts it into pigeonholes, sends copies to head office by a chosen courier, and clears out old post.

What is a daemon?

Programs that work in the background

- A **daemon** is a program that starts with the computer and keeps running in the background, with no window and nobody typing into it.
- It waits for something to happen, then does its job.
- Examples on Aldermoor's server: `sshd` waits for logins, `cron` runs scheduled jobs, `rsyslog` collects log messages.
- Their names often end in `d`, for daemon.

Syslog daemons

The application layer in practice

- **rsyslog**: the default on many Linux distributions, with rules, queues and TLS.
- **syslog-ng**: an alternative with similar features.
- **systemd-journald**: a binary journal on most current Linux systems, which can pass messages on to a syslog daemon.
- **Network devices** such as routers and firewalls send syslog directly to a configured collector.

How a daemon is told what to do

Configuration files

- A **configuration file** is a plain text file of instructions that a program reads when it starts.
- On Linux they live under `/etc`. rsyslog reads `/etc/rsyslog.conf`, and any files in `/etc/rsyslog.d/`.
- Usually **one instruction per line**. Lines starting with `#` are comments for people, and are ignored.
- After a change, the daemon must be **restarted** to read the file again.

Every message is labelled

Facility and severity, in brief

- Every syslog message carries two labels, which the daemon uses to decide where it goes.
- The **facility** says **what kind of program** sent it: the kernel, logins and security (`auth`, `authpriv`), background services (`daemon`), and so on.
- The **severity** says **how serious** it is, from `Debug` (lowest) to `Emergency` (highest).
- They are written together as `facility.severity`. For example, `auth.warning` means a security message at warning level.
- `*` stands for **any**, so `auth.*` means security messages at every level.

The full lists, and how the two labels become one number, come later in this part.

From messages to files

A typical Debian rsyslog configuration

- **Debian** is a widely used Linux operating system for servers. Ubuntu is built on it and behaves the same way. Aldermoor's server is set up like this.
- These two lines are **rules** from rsyslog's configuration (`/etc/rsyslog.conf`). Each rule says: messages like this go to that file.

```
auth,authpriv.*                 /var/log/auth.log
daemon.*                          -/var/log/daemon.log
```

- So `auth.log` holds the `auth` and `authpriv` messages, and `daemon.log` holds the `daemon` messages.
- The same message can match several rules, so it can go to several files, or to another machine.

Inside a rule

Selector, destination, and one small extra

In the rule	What it does
Left side, such as `auth,authpriv.*`	The **selector**, written `facility.severity`. A comma lists several facilities, and `*` means every severity.
Right side, such as `/var/log/auth.log`	The **destination**: the file that matching messages are added to.
A leading `-` before the file name	Do not force each message onto the disk at once. Faster, but a crash can lose the last few lines.

Both lines use the same pattern. Next, one rule read step by step.

Reading a rule

Step by step

Part	What it means
`auth,authpriv`	Which **facilities**: messages about logins and privileges
`.`	Separates the facility from the severity
`*`	Which **severities**: all of them
`/var/log/auth.log`	Where to **write** them: add each one to the end of this file

In one sentence: take every login and privilege message, however urgent, and add it to the end of `/var/log/auth.log`.

Where Linux keeps its logs

The folder `/var/log`

File	What goes in it
`auth.log`	Logins and `sudo` (Debian and Ubuntu)
`secure`	The same, on Red Hat systems
`daemon.log`	Background services
`syslog`, `messages`	Most other messages
`kern.log`	The kernel, the core of the operating system
`journal/`	systemd's own journal, read with `journalctl`

File names differ between systems: always check the configuration rather than assume.

Why forward at all?

Keeping copies somewhere else

- If a server is broken into, its own log files can be **changed or deleted**. A copy elsewhere cannot.
- One place to **search across many machines**, instead of logging in to each one.
- Logs **survive** when a machine fails or is rebuilt.
- Like a branch office sending copies of its records to head office every day.

Forwarding to a collector

rsyslog, the short form

```
# UDP: one @
*.*  @logs.aldermoor.example:514
# TCP: two @
*.*  @@logs.aldermoor.example:514
```

Reading the short form

Step by step

Part	What it means
`*.*`	Every facility, every severity: **everything**
`@`	Send it over **UDP**: send and forget
`@@`	Send it over **TCP**: check that it arrived
`logs.aldermoor.example`	The **collector**'s name
`514`	The **port**: the numbered door the collector listens at

Forwarding to a collector

rsyslog, done properly

```
action(type="omfwd" target="logs.aldermoor.example" port="6514"
    protocol="tcp"
      StreamDriver="gtls" StreamDriverMode="1"
    StreamDriverAuthMode="x509/name"
      queue.type="LinkedList" queue.filename="fwd_logs"
      queue.saveOnShutdown="on" action.resumeRetryCount="-1")
```

Forwarding to a collector

Line by line

Line	In plain words
1	Send messages to `logs.aldermoor.example`, at door 6514, over TCP.
2	**Encrypt** the connection (TLS), and check the collector's certificate, like checking an ID card before handing over a parcel.
3	Keep a **queue** of messages waiting to be sent, and store it on **disk** if it grows or the collector cannot be reached.
4	Keep the queue through a **restart**, and **keep retrying** rather than giving up.

Altogether: send every message to the collector, encrypted, and keep it safe on disk until it gets there. You do not need to remember the parameter names.

Transports

How messages travel

Transport	Standard	Delivery	Protection
UDP, port 514	RFC 5426	Best effort: lost messages go unnoticed	None: readable, and easy to spoof
TCP	RFC 6587	Reliable while connected	None
TLS, port 6514	RFC 5425	Reliable while connected	Encrypted, and certificates authenticate the hosts
Signed syslog	RFC 5848	Over any transport	Each message signed: origin and integrity, but rarely deployed

Miao, Ma and Salowey (2009) RFC 5425: TLS Transport Mapping for Syslog.
https://www.rfc-editor.org/rfc/rfc5425

Transports

In plain terms

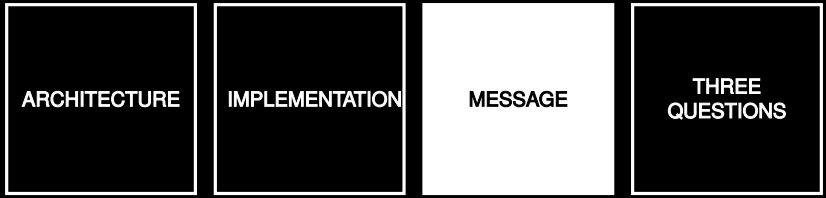
- **UDP** is a postcard: cheap and quick, anyone handling it can read it, and nobody tells you if it is lost.
- **TCP** is tracked post: you know it arrived, but it is still readable on the way.
- **TLS** is a sealed, tracked parcel, handed only to a recipient who has shown identification.
- **Signed syslog** is a wax seal on every letter: anyone can check who sent it and that nobody opened it.

Housekeeping that affects evidence

- **Rate limits**: journald and rsyslog drop messages from a program that logs too fast.
- **Rotation**: logrotate renames and compresses log files, and deletes the oldest after a set number of rotations.
- **Retention**: decide how long each log is kept before an incident, not after. At Target, the attack ran for weeks.
- **Clocks**: NTP keeps hosts in step, so events from different machines can be put in order.

Implementation
In short

- A **syslog daemon** on each machine sorts messages into files.
- It can also **forward** copies to a collector.
- **How** it forwards matters: UDP can lose and expose messages, and TLS protects them on the way.
- Settings such as **rate limits** and **rotation** decide what survives.


ARCHITECTURE
IMPLEMENTATION
MESSAGE
THREE QUESTIONS